---

title: Aura Virtual Try-On SaaS Platform subtitle: Comprehensive Technical Architecture and Design Report author: Aura Engineering Team

date: 2026-09-03

1. Overview of the Application

Aura is a highly scalable, multi-tenant Software-as-a-Service (SaaS) platform tailored specifically for the e-commerce fashion industry. It empowers independent fashion brands, boutique owners, and retail enterprises to launch fully customized, white-labeled digital storefronts equipped with an advanced Artificial Intelligence (AI) powered Virtual Try-On (VTO) engine.

Built using Python, Django 6.1 (ASGI mode), WebSockets (Daphne), TailwindCSS, HTMX, and Vanilla JS, Aura provides an enterprise-grade B2B2C solution out of the box.

1.1 The Core Proposition and Problem Statement

Traditional e-commerce fashion retail suffers from exceptionally high return rates. Aura solves this fundamental problem by allowing shoppers to upload a photo of themselves—creating a secure "Fit Passport." Using state-of-the-art diffusion models (like IDM-VTON, Fashn AI, and MobileVTON), the platform realistically renders clothing onto the customer's photo, accurately preserving fabric draping, textures, and lighting.

1.2 Multi-Tenant SaaS Capabilities

The platform operates as a massive multi-tenant system where a single unified codebase serves thousands of independent brands. - **Custom Domains & White-Labeling:** Brands automatically receive their own subdomain (e.g., `mybrand.yourdomain.com`) or can connect a fully custom domain powered by wildcard DNS and intelligent Django middleware. - **Dynamic Storefronts:** Brands fully customize their storefronts through a drag-and-drop JSON-based layout editor, establishing complete visual identity isolation. - **Fit Passport & Size Intelligence:** Customers set their height, weight, and fit preferences. Aura's algorithm intelligently recommends the perfect size for every product. - **One-Click Cloud Deployment:** Complete deployment infrastructure supporting DigitalOcean, AWS, or any VPS using Coolify with Docker. It includes production Dockerfiles, Traefik reverse proxy labels, and automatic Let's Encrypt SSL.

2. Project Plan

The development lifecycle of the Aura platform was meticulously structured using an Agile methodology.

Phase 1: Requirements Gathering & AI Prototyping

- **Objective:** Validate Generative AI models for virtual try-on (VTON) and define multi-tenant constraints.
- **Deliverables:** Integrated support for multiple VTON APIs including Hugging Face (IDM-VTON), Replicate API, and a Local Hardware Engine (MobileVTON) built on PyTorch.

Phase 2: Core Architecture & Multi-Tenancy

- **Objective:** Establish the Django backend, PostgreSQL database, and the crucial multi-tenant routing middleware.
- **Deliverables:** Implementation of `CustomDomainMiddleware` which dynamically maps incoming request hosts to internal brand slugs, bypassing the need for complex Nginx virtual hosts for every tenant.

Phase 3: E-Commerce & WebSockets Integration

- **Objective:** Develop shopping mechanisms and connect the Python backend to the asynchronous AI Generation engine using WebSockets.
- **Deliverables:**
  - Complete catalog and order management.
  - Setup of Redis/Celery queue for VTO image synthesis.
  - Implementation of Daphne ASGI to stream real-time generation progress to the UI.

Phase 4: Frontend UI/UX Development

- **Objective:** Design responsive dashboards for Platform Admins, Brand Owners, and Consumers.
- **Deliverables:**
    - Tailwind CSS + HTMX based frontend for blazing fast performance without heavy SPA JavaScript frameworks.

Phase 5: Deployment Strategy

- **Objective:** Ensure the system deploys across multiple hosting environments (Coolify, VPS, cPanel).
- **Deliverables:** Comprehensive deployment scripts, Dockerfiles, and Nginx/Apache configuration blocks for varied server infrastructures.

3. UML Diagrams

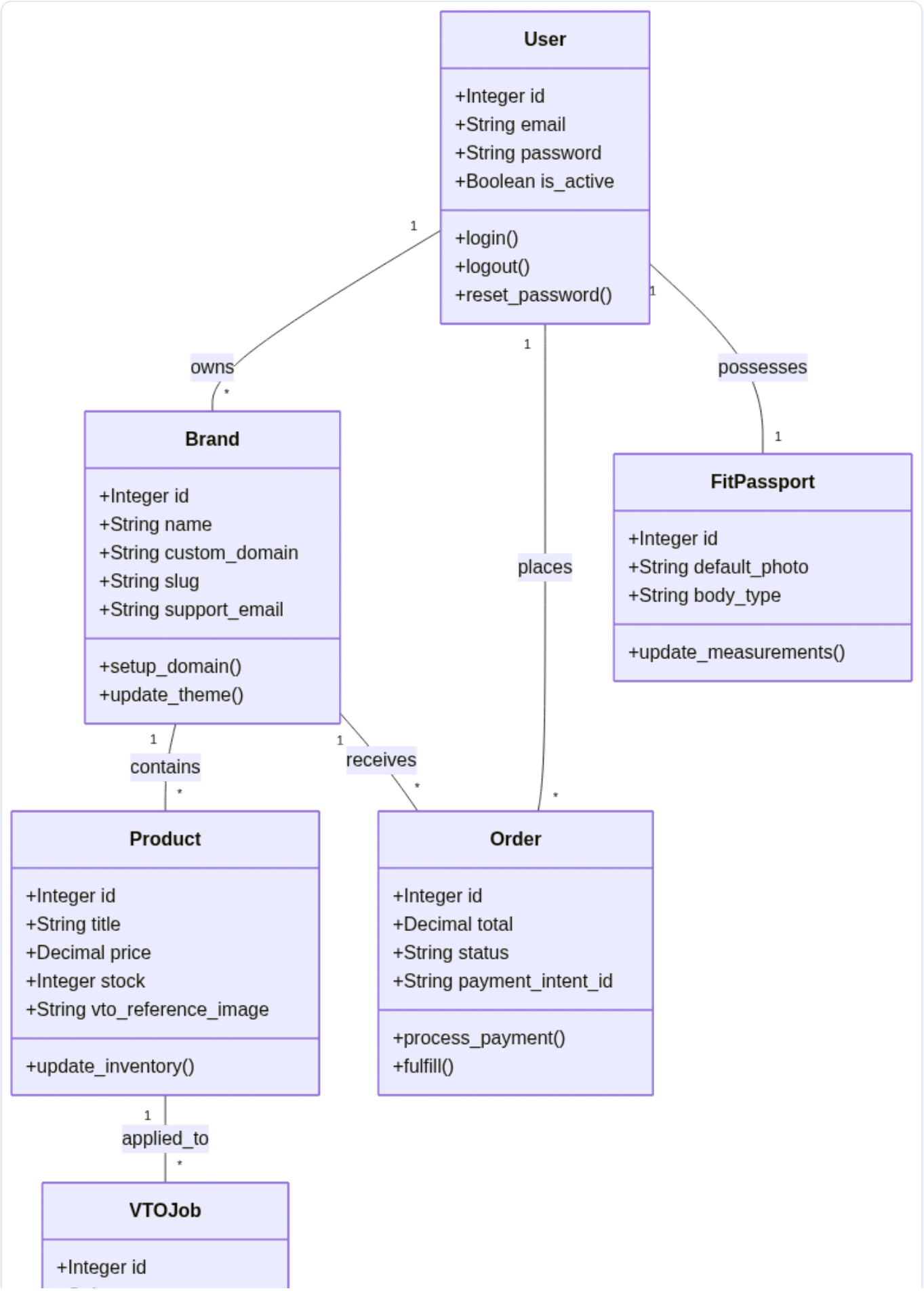
The following Unified Modeling Language (UML) diagrams provide a rigorous structural and behavioral blueprint of the Aura platform.

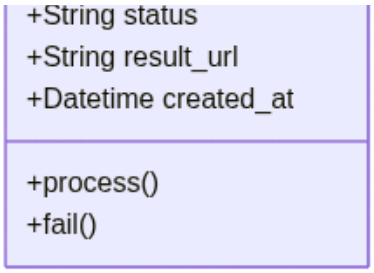
3.1 Use Case Diagram

The Use Case Diagram defines the critical interactions between the primary actors (Customer, Guest, Brand Owner, and Platform Admin) and the system.

3.2 Class Diagram

The Class Diagram illustrates the structural entities of the object-oriented Django backend. It showcases the attributes, methods, and relationships between the primary classes: Users, Brands, Products, Orders, and VTOJobs.


User
+Integer id
+String email
+String password
+Boolean is_active
+login()
+logout()
+reset_password()
owns
possesses
places
Brand
+Integer id
+String name
+String custom_domain
+String slug
+String support_email
+setup_domain()
+update_theme()
FitPassport
+Integer id
+String default_photo
+String body_type
+update_measurements()
contains
receives
Product
+Integer id
+String title
+Decimal price
+Integer stock
+String vto_reference_image
+update_inventory()
Order
+Integer id
+Decimal total
+String status
+String payment_intent_id
+process_payment()
+fulfill()
applied_to
VTOJob
+Integer id

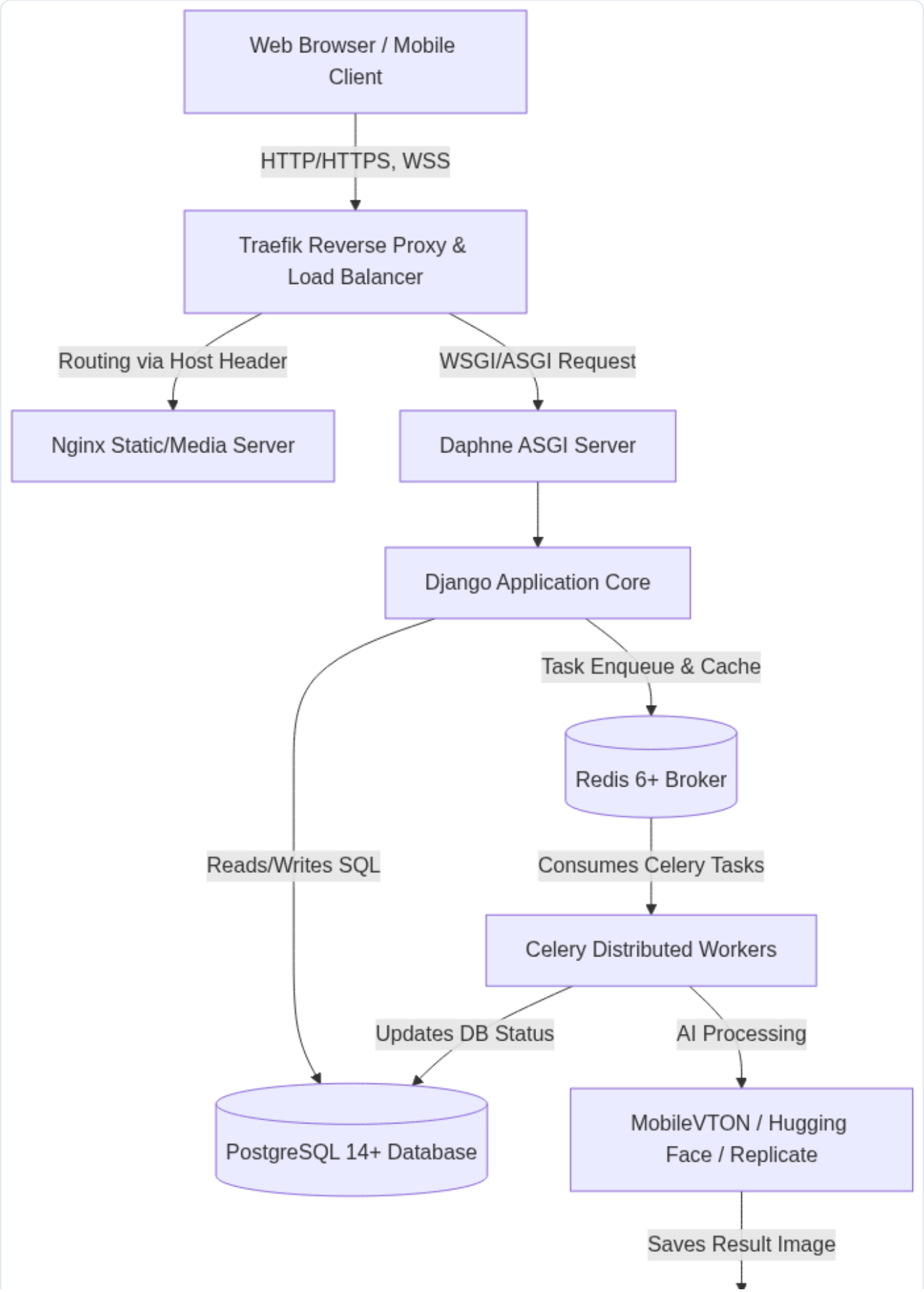

+String status
+String result_url
+Datetime created_at
+process()
+fail()

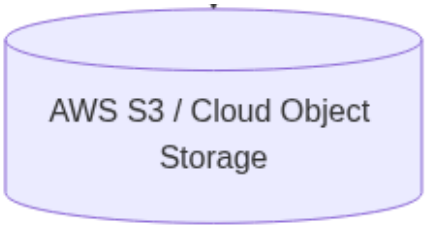
4. System Architecture

Aura utilizes a distributed architecture designed to seamlessly handle standard e-commerce HTTP traffic alongside computationally intensive AI tasks.

4.1 System Architecture Diagram

This diagram maps the flow of data from the client, through the load balancers, into the web application, and to the GPU workers.


Web Browser / Mobile Client
HTTP/HTTPS, WSS
Traefik Reverse Proxy & Load Balancer
Routing via Host Header
WSGI/ASGI Request
Nginx Static/Media Server
Daphne ASGI Server
Django Application Core
Task Enqueue & Cache
Redis 6+ Broker
Reads/Writes SQL
Consumes Celery Tasks
Celery Distributed Workers
Updates DB Status
AI Processing
PostgreSQL 14+ Database
MobileVTON / Hugging Face / Replicate
Saves Result Image



4.2 Description of the Application Layers

4.2.1 Presentation Layer (Frontend / Views)

- **Storefront Rendering:** Rendered server-side using Django Templates styled with Tailwind CSS and HTMX for rapid client-side transitions.
- **Real-Time Communication (ASGI):** Aura uses WebSockets via Django Channels (running on a Daphne ASGI server) to stream real-time AI progress updates directly to the client UI.

4.2.2 Business Logic Layer (Controllers / Middleware)

- **Multi-Tenant Routing (CustomDomainMiddleware):** This intercepts every incoming request, inspects the HTTP `Host` header, and dynamically rewrites the URL to serve the correct tenant's data (e.g., `shop.mybrand.com` routes internally to `brand_id=4`).
- **AI Task Delegation:** Validates image inputs, creates a `VTOJob`, and pushes a serialized job payload to the Redis message broker.

4.2.3 Data Access Layer (Models / ORM)

- **Django ORM & PostgreSQL 14+:** The primary database, ensuring ACID compliance and utilizing JSONB fields for dynamic storefront layouts.
- **Redis 6+:** In-memory caching and message broker for WebSockets.

4.2.4 Asynchronous Processing Layer (Workers / GPU)

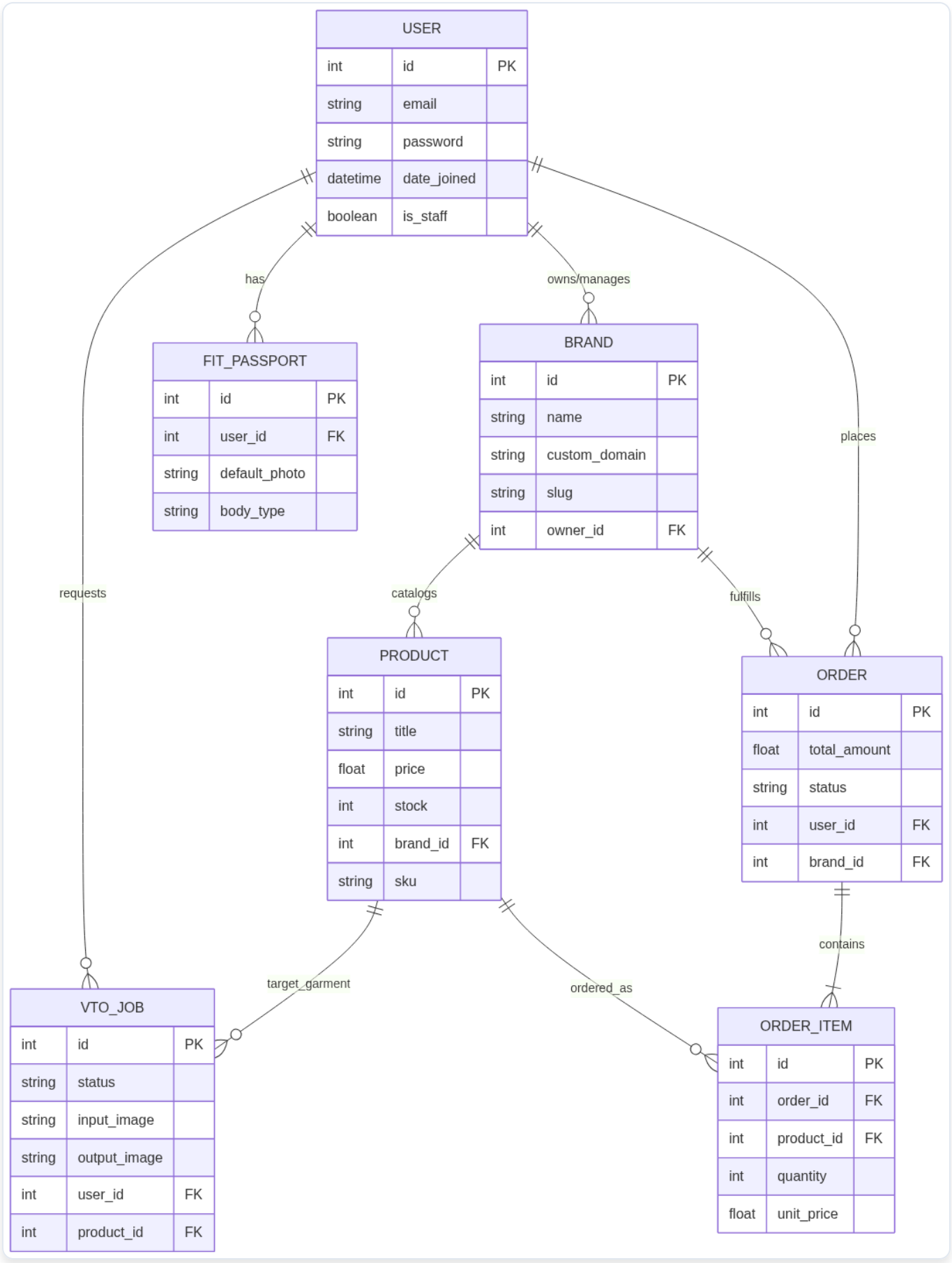
- **Celery Workers:** Consume tasks from Redis and interface with the AI engine.
- **AI Engines:** The platform supports abstract backend selection:
    1. **Hugging Face:** For cloud API processing via tokens.
    2. **Replicate API:** For paid, serverless GPU infrastructure.
    3. **Local Hardware Engine:** For running MobileVTON natively via local PyTorch/CUDA.

5. Database Design

The database enforces strict referential integrity. Tenant isolation is achieved via strict foreign key associations (`brand_id`) appended to all operational tables.

5.1 Entity-Relationship (E-R) Diagram

The following Entity-Relationship Diagram illustrates the logical schema and the cardinality of relationships.


USER
int id PK
string email
string password
datetime date_joined
boolean is_staff
has
owns/manages
places
requests
FIT_PASSPORT
int id PK
int user_id FK
string default_photo
string body_type
BRAND
int id PK
string name
string custom_domain
string slug
int owner_id FK
catalogs
fulfills
PRODUCT
int id PK
string title
float price
int stock
int brand_id FK
string sku
ORDER
int id PK
float total_amount
string status
int user_id FK
int brand_id FK
contains
target_garment
ordered_as
VTO_JOB
int id PK
string status
string input_image
string output_image
int user_id FK
int product_id FK
ORDER_ITEM
int id PK
int order_id FK
int product_id FK
int quantity
float unit_price

5.2 Description of Each Table

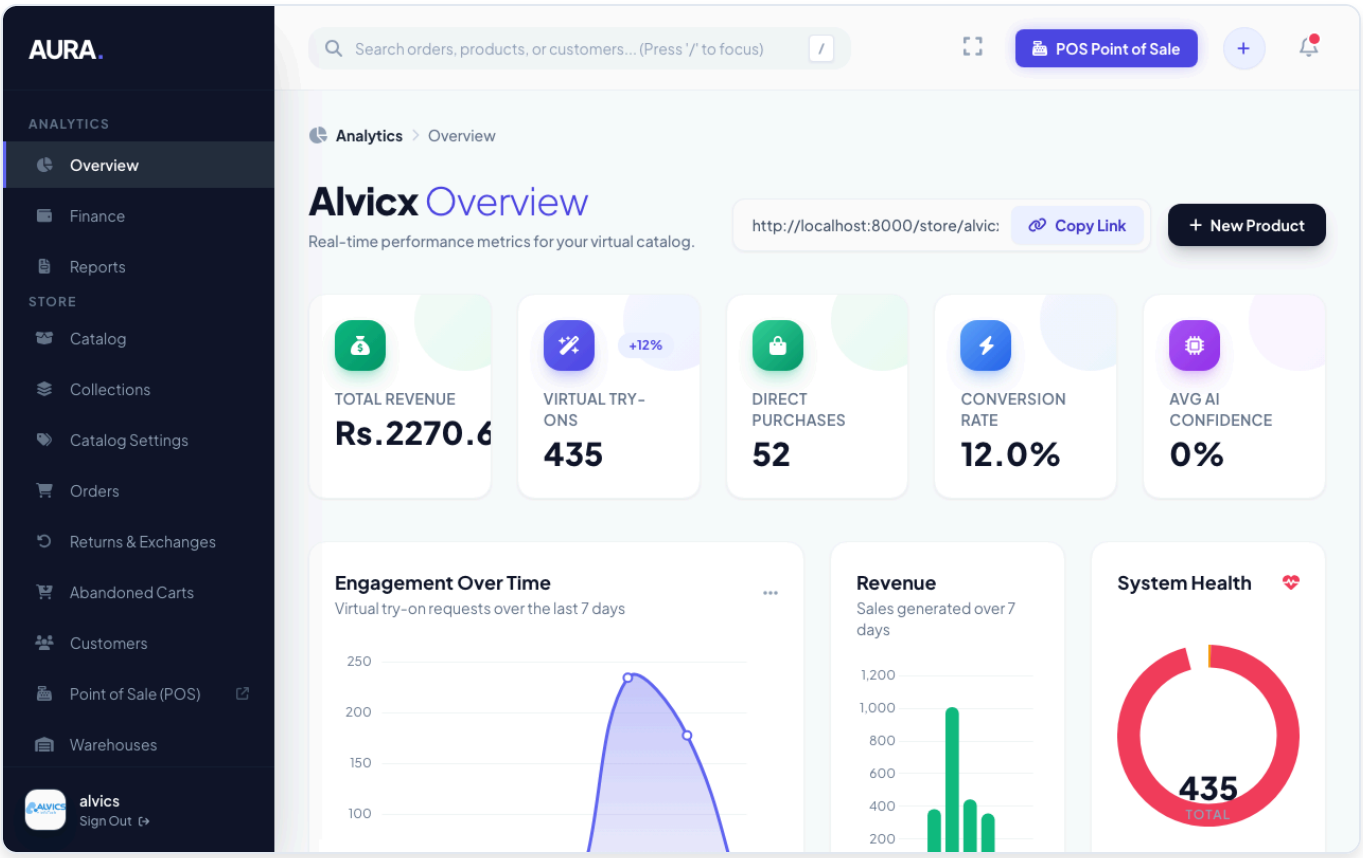
- **`auth_user`:** The central authentication table for all human entities.
- **`brands_brand`:** The core tenant record containing `custom_domain` and `slug`.
- **`catalog_product`:** Stores garments available for purchase. Includes `vto_reference_image`, a specialized masked image path used exclusively by the AI engine.
- **`orders_order` & `orders_orderitem`:** Records finalized transactions, bridging users, brands, and products.
- **`fitting_vtojob`:** Tracks the inputs (`input_image`), outputs (`output_image`), and status of asynchronous AI tasks.
- **`fitting_fitpassport`:** A reusable profile containing a user's bodily measurements and default photos.

6. User Manual

6.1 Brand Owner (Tenant) Manual

6.1.1 Brand Dashboard Overview

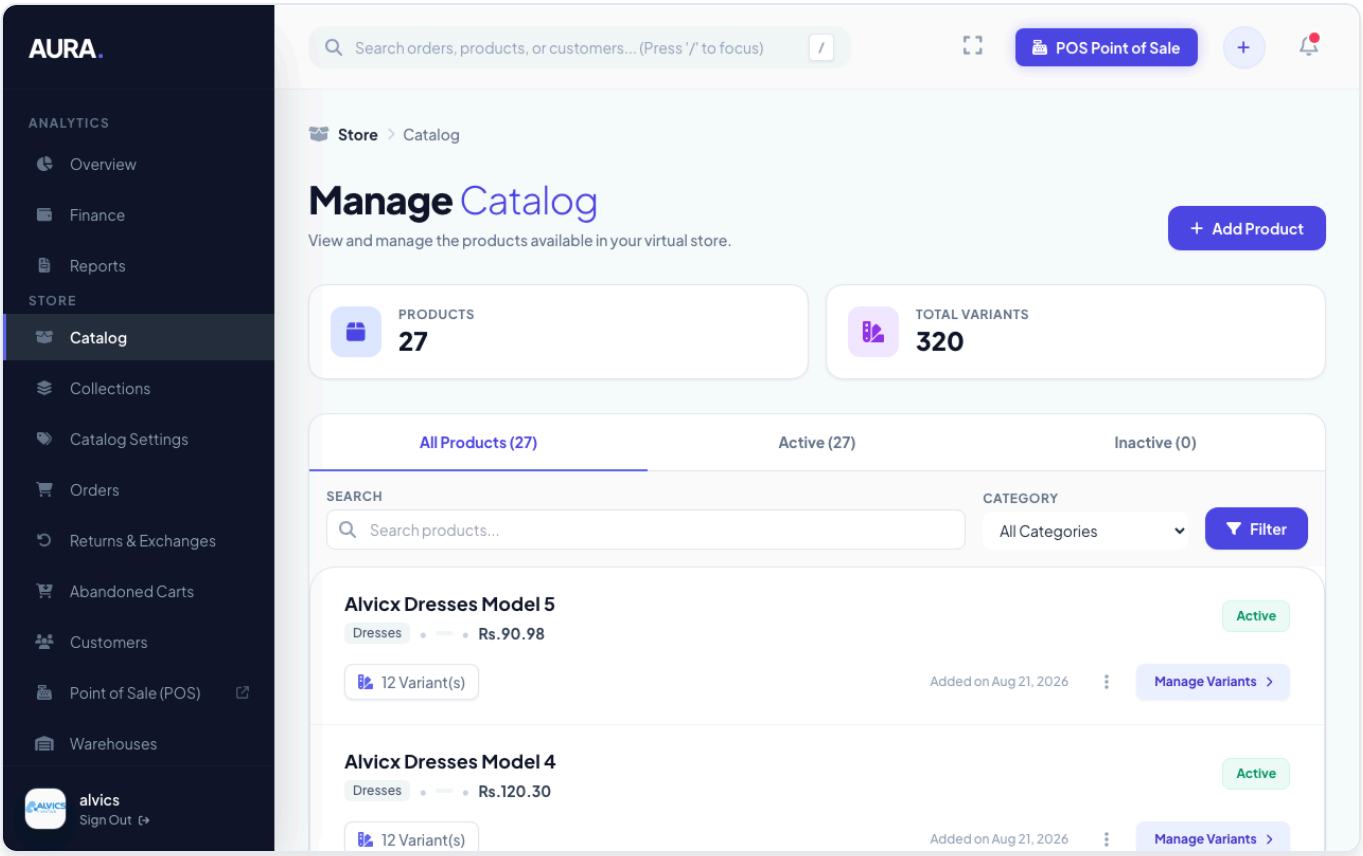
Upon logging in, brand owners access a unified dashboard displaying Gross Sales, Total Order Volume, and Virtual Try-On Engagement Metrics over the last 30 days.



6.1.2 Virtual Try-On Catalog Configuration

1. Navigate to **Catalog > Products** and click **Add Product**.
2. Upload standard product images.
3. In the Media section, upload a high-resolution, front-facing image of the garment on a flat surface or ghost mannequin.

4. Check **"Enable VTO"**. The system auto-generates a segmentation mask.



6.1.3 Customizing Your Storefront Domain

1. Go to **Settings > Custom Domain**.
2. Enter your desired domain (e.g., `shop.mybrand.com`).
3. Create a **CNAME record** in your DNS pointing to the platform domain. Traefik automatically issues an SSL certificate.

6.2 Customer Manual

1. Browse products on the brand's custom URL.
2. Click the **"Try It On (AI)"** button on a product page.
3. Upload a front-facing photo (saved to your **Fit Passport** if registered).
4. Within 10-15 seconds, the AI-generated image appears.

Page not found (404)

No Brand matches the given query.

Request Method: GET
Request URL: http://localhost:8000/store/owner-brand/
Raised by: apps.brands.views.storefront_view

Using the URLconf defined in `config.urls`, Django tried these URL patterns, in this order:

1. admin/ [namespace='admin']
2. django-admin/ [namespace='django_admin']
3. api/
4. p/
5. dashboard/billing/ [name='owner_billing']
6. dashboard/billing/checkout/<int:plan_id>/ [name='create_checkout_session']
7. api/billing/webhook/ [name='stripe_webhook']
8. [name='index']
9. dashboard/ [name='dashboard']
10. dashboard/settings/ [name='brand_settings']
11. dashboard/settings/team/ [name='team_management']
12. store/<slug:slug>/ [name='storefront']

The current path, `store/owner-brand/`, matched the last one.

You're seeing this error because you have `DEBUG = True` in your Django settings file. Change that to `False`, and Django will display a standard 404 page.

7. Application Installation Manual

This section details the rigorous steps required for System Administrators to deploy the Aura platform across various server environments including cPanel, VPS, and Coolify.

7.1 Server Requirements

Virtual Try-On models rely heavily on background tasks and WebSockets. - **OS:** Ubuntu 20.04/22.04 LTS or cPanel with Python Selector. - **Python:** Version 3.12+. - **Database:** PostgreSQL 14+ or MySQL 8.0+. - **Memory:** Minimum 4GB+ RAM for production. - **Cache/PubSub:** Redis Server 6.0+ (Required for WebSockets). - **ASGI Server:** Daphne or Uvicorn (Gunicorn alone will not support WebSockets).

7.2 AI Engine Configuration

Aura supports multiple backend engines accessible via the Global Config in the Admin Dashboard. 1. **Hugging Face (Default):** Communicates with open-source models (e.g., `yisol/IDM-VTON`). Requires a Hugging Face Read Token. (Note: High traffic requires a Pro account to bypass ZeroGPU limits). 2. **Replicate API:** A paid, serverless GPU infrastructure. Enter your API Key and select the model version (e.g., `cuiaxioming/idm-vton`). 3. **Local Hardware Engine (MobileVTON):** Runs inference directly on your server using PyTorch. **Warning:** Requires a dedicated NVIDIA GPU (Tesla T4 or better) to prevent out-of-memory crashes or 5+ minute generation times.

7.3 Custom Domain & Wildcard Infrastructure

Aura's `CustomDomainMiddleware` automatically routes subdomains and custom domains. - **DNS Configuration:** Create a Wildcard A Record (`*.yourdomain.com`) pointing to your server IP.

Traefik v3 Configuration (Coolify / Docker)

If using Coolify (which uses Traefik v3), configure the container labels: `text traefik.enable=true traefik.http.routers.http-0-`

`APP.rule=Host(`yourdomain.com`) && PathPrefix(`/`) traefik.http.routers.http-1-APP.rule=HostRegexp(`^[a-zA-Z0-9-]+\.yourdomain\.com$`) traefik.http.routers.https-0-APP.tls.certresolver=letsencrypt` *Note: In Traefik v3, HostRegexp uses standard Go regexp syntax. Ensure the "Escape special characters" checkbox is UNCHECKED in Coolify to prevent double-escaping.*

Nginx Configuration (Manual VPS)

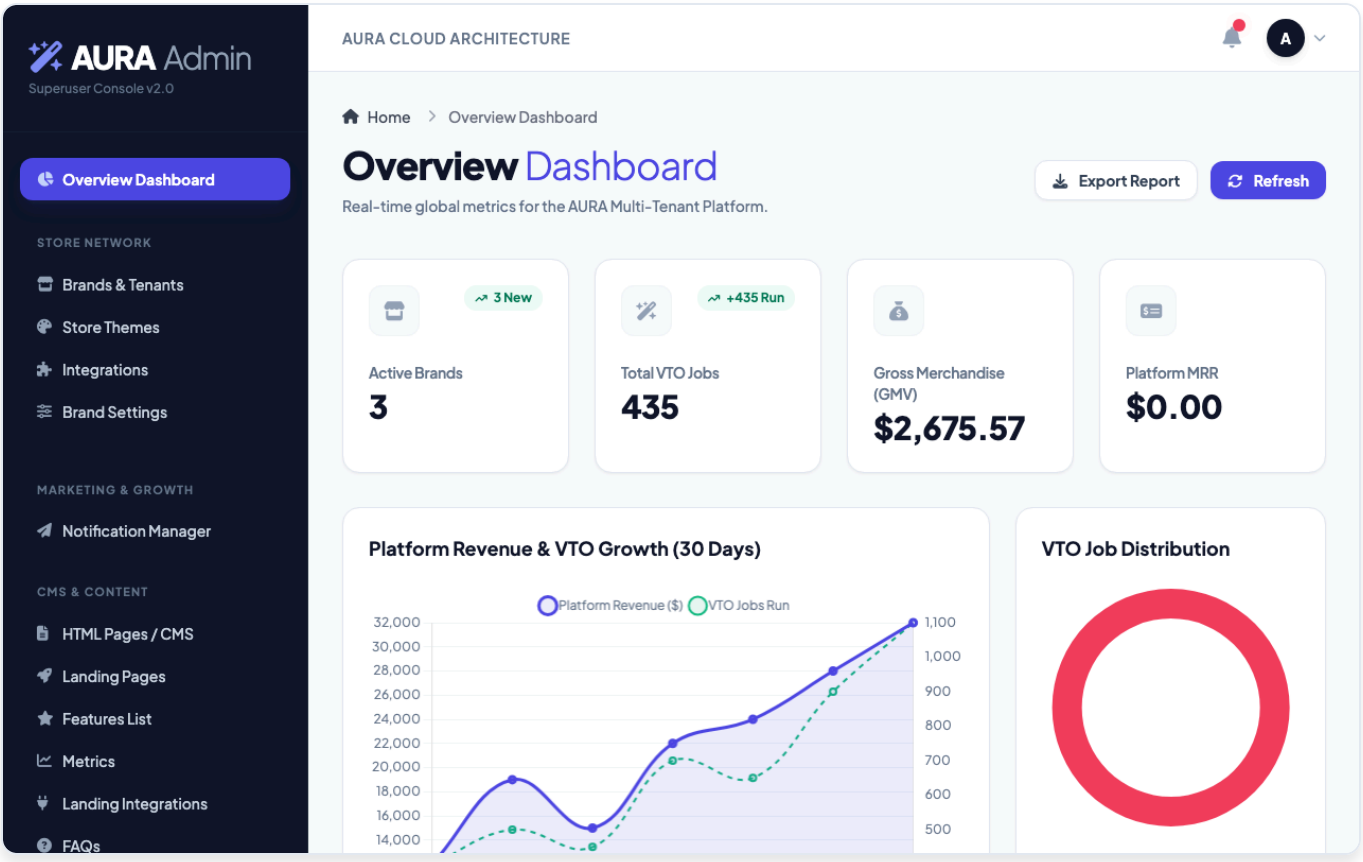
```
nginx server { listen 80; server_name yourdomain.com *.yourdomain.com; location / { proxy_pass http://127.0.0.1:8000; proxy_set_header Upgrade $http_upgrade; proxy_set_header Connection "upgrade"; proxy_set_header Host $host; } }
```

7.4 Deployment: cPanel (Python App)

cPanel uses Apache/Passenger. Because Aura requires WebSockets (ASGI), your cPanel host must support long-polling or ASGI fallbacks. 1. Navigate to **Setup Python App** and create an app (Python 3.10+). 2. Edit `passenger_wsgi.py` to point to the Django application: `python from config.wsgi import application` 3. **Domain Routing in cPanel:** - **Subdomains:** Create a subdomain (e.g., `brand1.yourdomain.com`) and point its Document Root to the Aura installation directory. - **Wildcards:** Edit the DNS Zone to add * A record, and create a * subdomain pointing to the Aura root. - **Addon Domains:** Add `mybrand.com` as an Addon Domain pointing to the exact same Document Root. The middleware handles the rest.

7.5 Deployment: DigitalOcean & Coolify (Recommended)

1. Create a DigitalOcean Droplet (Ubuntu 22.04 LTS, min 4GB RAM).
2. Install Coolify via SSH: `curl -fsSL https://cdn.coollabs.io/coolify/install.sh | bash`.
3. In Coolify, create a Project, add PostgreSQL and Redis databases.
4. Add the Aura Application from Git using the **Dockerfile** build pack.
5. Link the databases to inject `DATABASE_URL` and `REDIS_URL`.
6. Add the Traefik Custom Domain labels. Coolify will automatically provision Let's Encrypt SSL certificates.



End of Technical Report Document